

Respostas e eventuais comentários.

NOME: SOLUÇÃO N° estudante: _____

Cotação máxima: 50 pontos ; peso na nota final da disciplina: 50%

Estrutura da prova: escolha múltipla

Utilização: para cada pergunta só há uma resposta correcta; indique-a (com a letra correspondente) na folha de respostas, completando uma tabela semelhante

Cotação: cada resposta certa vale 1 ponto; cada resposta errada vale – 0,5 ponto (note o sinal menos!); cada resposta ambígua, ininteligível ou não assinalada. Se desejar, poderá fazer um pequeno comentário em alguma pergunta que lhe pareça ambígua. No caso de a sua resposta ser considerada errada, o seu comen

1	2	3	4	5	6	7	8	9	10	11	12			13	14	15	16	17	18
											a	b	c						
B	A	B	A	C	A	B	B	C	C	A	A	B	A	B	A	B	C	C	C

19	20			21	22	23	24	25			26	27	28	29	30	31	32	33	34
	T1	T2	T3					a	b	c									
C	B	C	B	C	B	A	B	C	B	B	A	A	C	C	A	A	B	B	C

35			36			37	38	39	40
.1	.2	.3	a	b	c				
B	A	B	B	A	A	A	B	A	C

1. Nas operações efectuadas com o comando: “gcc -Wall -o exec prog.c” para a criação de um executável,

- A) foi utilizada uma biblioteca estática.
- B) foi utilizada uma biblioteca dinâmica.
- C) não foi utilizada qualquer biblioteca.

Solução: Opção B ☒

Justificação: Por *default*, o gcc inclui sempre a *standard library* e faz *dynamic link* desta. É possível verificar isto compilando um simples programa em C e depois verificar quais são as bibliotecas incluídas.

```
jpdias on THINKPAD-T in jpdia/Projects/sope_experiments → ./t
hello world
jpdias on THINKPAD-T in jpdia/Projects/sope_experiments → ldd t
linux-vdso.so.1 (0x00007ffcf95e3000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007fee2255a000)
/lib64/ld-linux-x86-64.so.2 (0x00007fee2275e000)
```

Como se pode ver pela imagem, ao chamar o utilitário “ldd”, conseguimos ver que a biblioteca dinâmica “libc.so.6” é incluída.

Sabemos que se trata de uma biblioteca dinâmica quando são ficheiros do tipo **.so**. E sabemos que se trata de uma biblioteca estática quando são ficheiros do tipo **.a**.

2. O utilitário **make** funciona mediante regras de dependência e de datação de ficheiros.

A) Certo! Essencialmente, é isso que traduz o funcionamento de **make**.

B) Certo! Mas tais regras só podem ser utilizadas no desenvolvimento de programas.

C) Errado! O que importa é o que estiver estipulado nas primeiras linhas da **Makefile**.

Solução: Opção A 

Justificação: O utilitário **make** funciona mediante regras de dependência. Ou seja, na primeira linha do **Makefile** temos um conjunto de “comandos” que devem estar presentes noutras linhas. Por exemplo,

```
all: main utils

main: main.c

    gcc -c main.c -o main

utils: utils.c

    gcc -c utils.c -o utils
```

Para executar “all”, é necessário que os objectos “main” e “utils” existam nas linhas seguintes. Logo, o objecto “all” está dependente da existência dos outros objetos.

O utilitário **make** funciona mediante datação de ficheiros. Ou seja, o exemplo acima mencionado executará sempre os ficheiros **main.c** e **utils.c** mais recentes.

A opção B está incorreta, tais regras são utilizadas no ficheiro **Makefile** e não nos programas em si.

A opção C está correta exceto pela primeira parte que diz “Errado!” pelas razões já mencionadas.

3. Shell, em sistemas operativos, é

- A) um mero programa muito utilizado em Unix.
- B) um programa que faz de interlocutor entre o utilizador e o sistema operativo.
- C) uma linguagem de programação compilada.

Solução: Opção B ☒

Justificação: Shell é um programa que captura comandos do teclado e passa-os ao sistema operativo para os executar, permitindo então a comunicação entre o utilizador e o sistema operativo. É importante também realçar que o terminal é um programa que abre uma janela e permite ao utilizador comunicar com a shell.

A opção A está incorreta, pois a shell é um programa utilizado em todos os sistemas operativos e não só no Unix.

4. Um sistema operativo é um “gestor de recursos”. Isso quer dizer que:

- A) permite, por exemplo, a partilha da memória disponível no computador.
- B) faz a gestão dos recursos que as aplicações lhe submeterem.
- C) é apropriado a programas gestão (de bases de dados, por exemplo).

Solução: Opção A ☒

Justificação: Um sistema operativo é um “gestor de recursos”, pois gere os recursos do hardware, observando que recursos os programas estão a usar, concebendo pedidos de alocação de recursos e mediando pedidos conflituosos entre diferentes programas e utilizadores. Portanto, permite a partilha de memória disponível no computador entre diferentes programas e/ou utilizadores.

A opção B está incorreta, pois o sistema operativo, embora aloque os recursos de hardware necessários solicitados pela aplicação, não gere os recursos submetidos por esta.

5. Sistemas multi-programação e sistemas multi-utilizador

- A) são sinónimos, pois têm o mesmo significado.
- B) são antagónicos, pois não podem ambos qualificar um dado sistema operativo.
- C) são coisas diferentes, apesar de normalmente virem associados.

Solução: Opção C ☒

Justificação: Um sistema multi-programação é um sistema que permite a ocorrência de vários processos “em simultâneo”. O CPU troca de processo muito rapidamente dando a ilusão de que ocorrem ao mesmo tempo. No entanto, a cada instante, o CPU apenas corre um processo de cada vez.

Um sistema multi-utilizador, como o próprio nome indica, é um sistema que permite o acesso ao computador por múltiplos utilizadores.

Portanto, são coisas diferentes que normalmente vêm associadas, pois havendo múltiplos utilizadores num sistema é necessário que haja também múltiplos processos a ocorrer em simultâneo. Cada utilizador com os seus próprios processos.

6. Num sistema operativo, uma “chamada ao sistema”

- A) é a maneira como um sistema operativo aceita pedidos.
- B) é a maneira de se comunicar com um sistema do tipo Unix.
- C) é a maneira como, num terminal, um utilizador pede um recurso.

Solução: Opção A ☒

Justificação: O terminal permite que o utilizador comunique com a shell, que por sua vez comunica com o sistema operativo, através de “systems calls” ou “chamadas ao sistema”. Portanto, é através desta chamada que o sistema operativo interpreta e aceita/recusa pedidos do utilizador.

A opção B está incorreta, pois esta é maneira de se comunicar com qualquer sistema operativo, não só um sistema do tipo Unix.

A opção C não está inteiramente correta, pois as chamadas ao sistema permitem fazer mais coisas para além de conceder acessos de recursos ao utilizador.

7. O barramento (bus) de um computador

- A) é o principal componente de computadores de elevado desempenho.
- B) permite a interligação dos dispositivos electrónicos do computador.
- C) é um dispositivo do tipo Entrada/Saída (I/O).

Solução: Opção B ☒

Justificação:

8. Um processo tem associado, pelo menos:

- A) um programa, uma zona de memória partilhada e três ficheiros (`stdin`, `stdout`, `stderr`).
- B) um programa, uma zona de memória e uma entrada na tabela de processos.
- C) um programa, um processador e um dono.

Solução: Opção B ☒

Justificação: Um processo é uma instância de um programa em execução. Logo, deve ter pelo menos um programa associado.

Para além disso, deve ter também uma zona de memória para guardar as informações relativas a esse processo. Essa zona está disponível apenas para esse processo e os seus threads e, portanto, a opção A está incorreta. Caso o processo queira comunicar com outro processo, fá-lo através de mecanismos de comunicação como *pipes* e não partilhando a sua zona de memória.

Por fim, deve ter também uma entrada na tabela de processos. A tabela de processos possui uma entrada por processo. Essa entrada possui informação importante que deve ser guardada quando o processo troca de estado (*ready*, *running* ou *blocked*) para que possa ser reiniciada depois. Os três ficheiros *stdin*, *stdout*, *stderr* são apenas um exemplo de informação guardada nessa entrada.

Os processos têm um dono associado, como se pode verificar com o utilitário “top”. Tal deve-se ao facto de estarem associados a um programa. Sendo um programa um ficheiro normal com permissões (e, portanto, um dono) também o processo tem um dono associado.

Um processo precisa também de um processador para que possa correr, mas não precisa de estar associado a este, pois os processos podem estar em memória sem usarem o processador.

9. Na sequência da invocação da chamada `fork()`, o novo processo irá começar imediatamente a executar.

A) Não, pois o processo-pai é sempre o primeiro a executar.

B) Sim, se for escolhida a opção certa para `fork()`.

C) Não é possível afirmar isso com certeza.

Solução: Opção C ☒

Justificação: O comando `fork()` gera um novo processo, que continuará a execução do programa a partir da invocação desse comando. Efetivamente, passará a ter 2 processos em execução após invocar a função `fork()`: o processo filho e o processo pai. Se é o processo-pai ou se é o processo-filho quem executa primeiro não se sabe. Quem executa varia, embora possa parecer que é sempre um quem começa. E, portanto, não se pode afirmar com certeza qual dos processos começará a executar.

10. Considere o excerto de código junto. O que será impresso na saída padrão?

```
int global = 0;
void main() {
    int pid = fork();
    if (pid == 0) { global++; }
    else { wait(NULL); global--; }
    printf("%d ", global);
}
```

A) -1 1

B) 0 0

C) 1 -1

Solução: Opção C ☒

Justificação: Inicialmente, a variável “global” está a 0. Depois, é efetuado um `fork()` que criará um novo processo e, portanto, haverá dois processos em execução. Dentro do `if`, as instruções serão executadas pelo processo-filho. Dentro do `else` serão executadas pelo processo-pai. A chamada `wait()` faz com que o processo-pai espere que um qualquer processo-filho termine, antes de executar a seguinte instrução.

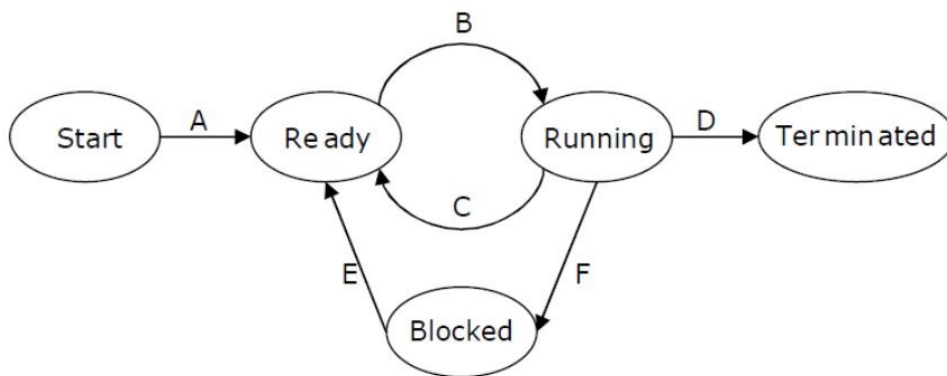
Logo, o processo filho coloca a variável global a 1 e o processo pai coloca a variável a -1. Devido à chamada `wait()` podemos garantir que o processo filho é executado primeiro. Daí que a opção C seja a correta e não a opção A.

11. Os três principais estados de um processo reconhecido pelo sistema operativo são:

- A) prontidão (ready), bloqueamento (blocked), execução (running).
- B) dormência (sleeping), bloqueamento (blocked), execução (running).
- C) prontidão (ready), bloqueamento (blocked), sinalização (signaling).

Solução: Opção A ☒

Justificação:



12. Considere o seguinte excerto de código, relativo ao tratamento de sinais. O programa é executado e não há erros na invocação dos serviços do sistema.

```
void handler(int signo) { printf("Hello "); }

void main() {
    struct sigaction new;
    sigset_t smask; sigemptyset(&smask);
    new.sa_handler = handler; new.sa_mask = smask; new.sa_flags = 0;
    sigaction(SIGUSR1, &new, NULL);
    pause(); printf("World.\n");
}
```

O que aparecerá na saída padrão assim que for enviado ao processo o sinal

a) SIGUSR1?

- A) Hello World.
- B) World. Hello
- C) World.

Solução: Opção A ☒

Justificação: A função `sigaction()` apanha o sinal `SIGUSR1`, por isso o *handler* do sinal (função `handler()`) é executado.

b) SIGINT?

- A) [nada e o programa continua]
- B) [nada e o programa termina]
- C) World.

Solução: Opção B ☒

Justificação: O sinal SIGINT não é apanhado, logo a sua ação por defeito é executada que é terminar.

c) SIGCHLD?

- A) [nada e o programa continua]
- B) [nada e o programa termina]
- C) World.

Solução: Opção A ☒

Justificação: O sinal SIGCHLD não é apanhado, logo a sua ação por defeito é executada que é ignorar.

13. Deve optar-se por programação com threads relativamente à programação por processos,

- A) quando se pretender escrever um servidor.
- B) quando se pretender facilitar a troca de informação entre as partes do programa.
- C) quando se pretender aumentar o isolamento entre as partes do programa.

Solução: Opção B ☒

Justificação: Dentro do mesmo processo, as *threads* partilham o mesmo espaço de endereços facilitando a troca de informação entre as várias partes do programa/processo.

A opção C está incorreta. Quando se pretende aumentar o isolamento entre as partes do programa opta-se por criar vários processos independentes entre si e não *threads*.

A opção A pode ser verdadeira em alguns casos, mas não em todos os casos. Depende das especificações para cada servidor.

14. Na criação de um thread, com `pthread_create()`, terá de se especificar uma função que irá estar ligada à execução do novo thread.

A) É certo, mas essa função tem de ter um protótipo do tipo `void* function (void*)`.

B) É certo, mas essa função apenas necessita de receber como argumentos um apontador para void.

C) É certo, mas não há qualquer restrição ao tipo do valor de retorno da função

Solução: Opção A 

Justificação:

```
int pthread_create(pthread_t *thread, pthread_attr_t *attr, void *(*start_routine)(void *),  
void *arg);
```

A função que irá estar ligada à execução do novo thread é a especificada no terceiro parâmetro da função `pthread_create()`. E como se pode observar essa função deve sempre retornar e receber como parâmetro um apontador para `void*`.

15. Para se poder atender um grande número de processos, basta usar um quantum pequeno, que melhora a rotatividade de utilização do processador.

A) Isso é importante, especialmente porque o custo das mudanças de contexto é mínimo.

B) Isso parece precipitado, pois há que considerar o custo das mudanças de contexto.

C) Isso só pode ser feito nos sistemas de grande utilização do processador.

Solução: Opção B 

Justificação:

16. Uma condição de competição (race condition) surge quando dois ou mais processos

- A) tentam partilhar de forma eficiente um mesmo conjunto de recursos.
- B) tentam partilhar de forma sincronizada um mesmo conjunto de recursos.
- C) tentam partilhar de forma dessincronizada um mesmo conjunto de recursos.

Solução: Opção C ☒

Justificação: Uma condição de competição surge quando dois processos ou mais acedem a dados em comum e tentam mudá-la “ao mesmo tempo”. Isto acontece porque os processos tentam partilhar um mesmo conjunto de recursos sem qualquer tipo de sincronização. Estando os processos devidamente sincronizados, ou seja, sabe-se quem acede primeiro à região crítica e quem acede depois, as condições de competição dificilmente ocorrem.

17. Pode um processo fora de uma zona crítica impedir outro de entrar nessa zona?

- A) Pode, mas não deve, porque o mecanismo de reserva antecipada é complicado.
- B) Pode, e deve, pois isso até melhora a eficiência do sistema de exclusão mútua partilhada.
- C) Pode, mas não deve, pois estaria a tornar o sistema ineficiente e com tendência a encravar.

Solução: Opção C ☒

Justificação: Um processo fora da zona crítica pode, mas não deve, impedir outro processo de entrar nessa zona. Fazê-lo seria violar uma das condições para uma boa “mutual exclusion”: “No process running outside its critical region may block another process.” Para além disso, qual seria o objetivo de fazer isso?

18. Em programação POSIX, uma variável de condição deve ser sempre usada em associação com um mutex.

A) Não, tal não é necessário, pois a variável de condição é suficiente.

B) Talvez, se não se puder usar outro tipo de semáforo.

C) Sim, para garantir o acesso exclusivo à zona onde se vai usar a variável de condição.

Solução: Opção C ☒

Justificação: Uma variável de condição está sempre associada a um *mutex* e permitem que as threads bloqueiam devido a alguma condição.

19. Um encravamento (deadlock) típico dá-se quando vários processos de um grupo se vêm impossibilitados de prosseguir a sua actividade normal por

A) todos terem esgotado o seu quantum de processador.

B) todos estarem aguardando a libertação do processador.

C) todos necessitarem de recursos detidos por diferentes processos do grupo.

Solução: Opção C ☒

Justificação: *Deadlocks* acontecem quando todos os processos estão à espera uns dos outros para entrar na região crítica onde está outro processo.

Por exemplo, quando o processo 1 está na região crítica A e quer entrar na região crítica B. E o processo 2 está na região B e quer entrar na região A. Os dois processos ficam à espera que o outro saia da sua região (liberte o seu recurso). Como os dois ficam à espera e nunca nenhum sai dá-se um encravamento/*deadlock*.

20. Um programa é composto por 3 actividades (threads) , T1, T2 e T3 que, de vez em quando, invocam uma função `work(...)` cujos argumentos, em número variável, identificam os recursos necessários à sua execução, que devem ser acedidos de forma exclusiva.

Assim, por exemplo, T3 poderá a certa altura invocar `work(R2,R3)`, querendo isto dizer que deverá haver código que garanta que os recursos R2 e R3 são unicamente usados por T3 durante a execução da função. Uma forma de conseguir isso, usando 2 mutexes, m2 e m3, seria:

```
lock(m2); lock(m3); work(R2,R3); unlock(m2); unlock(m3);
```

Supondo que cada um de 3 recursos R1, R2 e R3 é protegido individualmente por um mutex, respectivamente, m1, m2 e m3, escolha, para cada um dos threads, a opção mais apropriada que impeça o programa de encravar, executando com o máximo de eficiência.

NOTA: tenha em atenção que o que escolher para um thread pode afectar a escolha para os outros!

T1 :	T2 :	T3 :
A) lock(m1); lock(m2); lock(m3); work(R1,R2); unlock(m3); unlock(m2); unlock(m1);	A) lock(m1); lock(m3); lock(m2); work(R1,R2,R3); unlock(m1); unlock(m3); unlock(m2);	A) lock(m3); lock(m2); work(R2,R3); unlock(m1); unlock(m3);
B) lock(m1); lock(m2); work(R1,R2); unlock(m2); unlock(m1);	B) lock(m3); lock(m1); lock(m2); work(R1,R2,R3); unlock(m1); unlock(m2); unlock(m3);	B) lock(m2); lock(m3); work(R2,R3); unlock(m3); unlock(m2);
C) lock(m2); lock(m1); work(R1,R2); unlock(m2); unlock(m1);	C) lock(m1); lock(m2); lock(m3); work(R1,R2,R3); unlock(m3); unlock(m2); unlock(m1);	C) lock(m3); work(R2,R3); unlock(m3);

Solução: Opções T1.B, T2.C, T3.B ☒

Justificação: Começemos por eliminar coisas que estão claramente erradas:

- T1.A está errado, porque não faz sentido usar os três mutexes se só queremos usar dois recursos. Só se deve fazer *lock* daquilo que se precisa.
- T3.A está errado, porque está a dar *unlock* de m1 quando nunca tinha dado *lock*. Não faz sentido libertar um recurso que nunca se chegou a ter.
- T3.C está errado, porque está a dar *lock* apenas do recurso 3 quando se precisa também do recurso 2. Originaria uma *race condition*.

T1 :	T2 :	T3 :
	A) lock(m1); lock(m3); lock(m2); work(R1,R2,R3); unlock(m1); unlock(m3); unlock(m2);	
B) lock(m1); lock(m2); work(R1,R2); unlock(m2); unlock(m1);	B) lock(m3); lock(m1); lock(m2); work(R1,R2,R3); unlock(m1); unlock(m2); unlock(m3);	B) lock(m2); lock(m3); work(R2,R3); unlock(m3); unlock(m2);
C) lock(m2); lock(m1); work(R1,R2); unlock(m2); unlock(m1);	C) lock(m1); lock(m2); lock(m3); work(R1,R2,R3); unlock(m3); unlock(m2); unlock(m1);	

Agora, imagine-se a seguinte situação:

A:	B:
lock(m1);	lock(m2);
lock(m2);	lock(m1);
f(...);	f(...);
unlock(m2);	unlock(m1);
unlock(m1);	unlock(m2);

O processo A faz *lock* do m1. Imagine-se que logo a seguir o sistema operativo decide que é a vez do processo B trabalhar. Então o B começa e pega no *lock* m2. Depois tenta ir buscar o m1 e bloqueia, porque alguém já tem esse recurso. Então, o sistema operativo passa a vez a outro *thread* até chegar a vez do A, que tenta pegar no m2, mas o B também já o tem, então o A também bloqueia, e passa a vez. A partir daqui, sempre que o sistema operativo der a vez a um destes dois threads, eles vão ficar bloqueados, porque não há nenhuma maneira de libertarem os *locks*. Ou seja, ficam num *deadlock*, continuam a existir mas na prática o programa já não consegue avançar mais.

Portanto, a ordem com que damos *lock* importa no sentido de evitar *deadlocks*. Nos *unlocks*, não é possível garantir nenhuma ordem (e também não é necessário).

Aplicando a mesma lógica ao exercício, ignore-se a existência do recurso 1 por um momento. Nos casos T2.A e T2.B, se ignorarmos o *lock* e *unlock* do m1, teremos uma situação muito parecida com a situação atrás mencionada. Ou seja, esses casos levariam a *deadlocks*.

T1 :	T2 :	T3 :
B) lock(m1); lock(m2); work(R1,R2); unlock(m2); unlock(m1); C) lock(m2); lock(m1); work(R1,R2); unlock(m2); unlock(m1);	C) lock(m1); lock(m2); lock(m3); work(R1,R2,R3); unlock(m3); unlock(m2); unlock(m1);	B) lock(m2); lock(m3); work(R2,R3); unlock(m3); unlock(m2);

Finalmente, compare-se o T1 com o T2 ignorando, desta vez o *lock* e *unlock* de m3. No caso do T1.C, teremos também uma situação parecida com a mencionada em cima, que levará a *deadlocks*.

T1 :	T2 :	T3 :
B) lock(m1); lock(m2); work(R1,R2); unlock(m2); unlock(m1);	C) lock(m1); lock(m2); lock(m3); work(R1,R2,R3); unlock(m3); unlock(m2); unlock(m1);	B) lock(m2); lock(m3); work(R2,R3); unlock(m3); unlock(m2);

21. A hierarquia de memória de um computador

A) consiste em registos de processador, zonas temporárias de sistema (buffers) e memória dinâmica.

B) existe porque se deseja usar vários tipos diferentes de componentes de memória.

C) é necessária porque não se consegue obter a baixo custo toda a memória rápida que se deseja.

Solução: Opção C ☒

Justificação:

22. Considere um sistema em que a memória, numa dada ocasião, apresenta os seguintes espaços vazios (holes), por ordem crescente de posição: 10KiB, 4KiB, 20KiB, 18KiB, 7KiB e 9KiB. Nessa situação, foram feitos 3 pedidos sequenciais de memória: 10KiB, 12KiB e 9KiB, após o que a lista ordenada de espaços vazios ficou: 4KiB, 20KiB, 6KiB e 7KiB. Qual foi o método de alocação utilizado?

- A) Primeiro a servir (first fit).
- B) Melhor a servir (best fit).
- C) Pior a servir (worst fit)

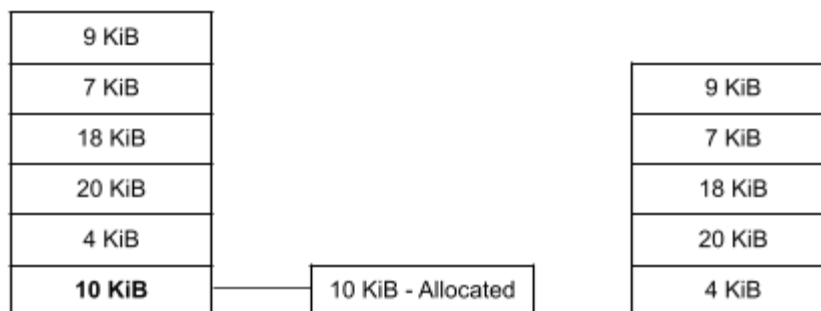
Solução: Opção B ☒

Justificação: A estratégia aqui passa por usar cada um dos métodos de alocação e ver em qual deles corresponde à versão final do enunciado.

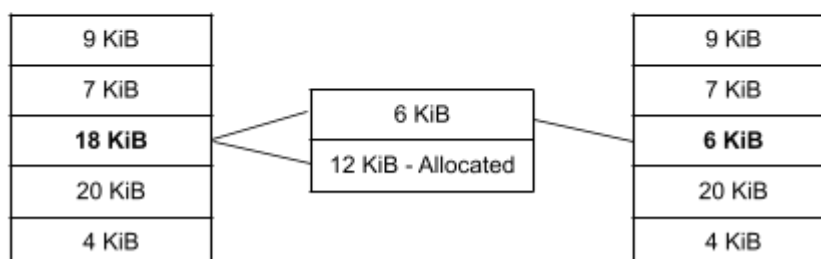
A opção correta B corresponde ao pedido como se pode ver de seguida.

“Best fit searches the entire list, from beginning to end, and takes the smallest hole that is adequate. Tries to find a hole that is close to the actual size needed, to best match the request and the available holes.”

1º



2º



3º



23. Uma entrada de uma tabela de páginas de um sistema de memória virtual contém, entre outros dados,

- A) um número de moldura (frame), um bit de presença e um bit de alteração.
- B) um número de moldura (frame), um número de página e um bit sinalização.
- C) um número de página, um bit de falha de página e um número de bloco de disco.

Solução: Opção A ☒

Justificação: A opção A é a correta como se pode ver na seguinte figura (do livro “Modern Operating Systems”, de Tanenbaum e Bos):

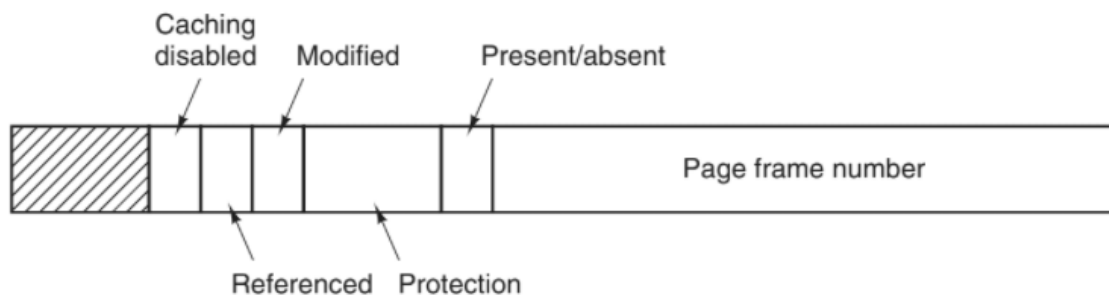


Figure 3-11. A typical page table entry.

A opção B está incorreta, pois o número da página está apenas presente na tabela das páginas (e serve como ID de cada entrada) e não na entrada em si. Para além disso, não existe um bit de sinalização.

A opção C está incorreta, pois também não existe um bit de falha de página nem um número de bloco de disco.

24. Num sistema de memória virtual, a escolha do algoritmo de substituição de página é especialmente importante por questões de

- A) economia do espaço de memória disponível.
- B) melhoria do desempenho do sistema.
- C) minimização do desgaste do disco de apoio (swap).

Solução: Opção B ☒

Justificação: A opção A indica-nos a razão pela qual optamos por *virtual memory* em vez de *swapping* ou *base and limit registers*. Esses três métodos respondem à questão de como dar a cada programa o seu próprio espaço de endereços e, portanto, como economizar o espaço da memória disponível.

Os algoritmos de substituição de página relacionam-se com o método de *virtual memory with paging*. Trata-se de vários algoritmos que tentam responder ao problema de que páginas devemos remover da memória para dar espaço a uma nova página (isto é, quando ocorre uma *page fault*). E consoante o algoritmo escolhido poderemos estar a melhorar ou a piorar o desempenho do sistema. Um algoritmo que prejudique este desempenho é um em que estas substituições de páginas ocorrem frequentemente. Um “bom” algoritmo que melhore o desempenho é um que consegue prever as páginas que ainda serão usadas e recorrer à substituição das páginas o mínimo possível.

25. Considere um sistema que implementa memória paginada com (apenas!) 4 quadros (page frames). A tabela junto mostra a informação de paginação acessível ao sistema operativo na altura em que uma decisão de libertação de quadro tem de ser tomada.

quadro	pág.	carregada há (clock ticks)	referenciada há (clock ticks)	R flag	M flag
0	25	136	122	1	0
1	81	290	260	0	0
2	19	145	145	1	1
3	97	280	250	0	1

Qual das páginas mostradas dará lugar à nova página, quando o algoritmo usado for o da "página que...

a) ...não é usada recentemente" (NRU, Not Recently Used)?

A) 19

B) 25

C) 81

Solução: Opção C ☒

Justificação: O que é pedido no enunciado é qual das páginas será removida da memória para dar lugar a uma nova página. Usando o algoritmo NRU, queremos eliminar a página que pertença à classe com o número mais baixo.

Classe	Significado	Páginas
3	referenciada e modificada	19
2	referenciada, mas não modificada	25
1	não referenciada, mas modificada	97
0	não referenciada, nem modificada	81

E, neste caso, a página 81 pertence à classe 0, e portanto, essa página não é referenciada nem modificada. Logo, essa é a página que será removida.

b) ...entra primeiro, sai primeiro" (FIFO)?

A) 25

B) 81

C) 97

Solução: Opção B ☒

Justificação: O algoritmo de FIFO mantém uma lista com todas as páginas em memória. No fim da lista temos a página que foi adicionada mais recentemente e no início da lista temos a página menos recente de todas.

A lista das páginas seria,

[81, 97, 19, 25]

Esta lista é obtida ordenando as páginas da tabela por ordem decrescente da coluna “carregada há”.

A página 81 foi a que foi carregada à mais tempo (290 *clock ticks*). Logo, essa é a página a remover.

c) ...entra primeiro, sai primeiro; com 2ª chance" (FIFO, second chance)?

A) 25

B) 81

C) 97

Solução: Opção B ☒

Justificação: O algoritmo FIFO, second chance corresponde ao algoritmo FIFO, mas com uma pequena modificação. Não só procura a página que foi carregada há mais tempo, como também procura pela página menos recente e que não foi referenciada.

Ou seja, enquanto que no algoritmo FIFO só olhamos para a coluna “carregada há”, neste algoritmo devemos olhar para essa coluna e para a coluna “R flag”. E verificamos, que a página 81 não só é a que foi carregada há mais tempo como também não foi referenciada (R=0).

26. Nos sistemas reais que usam paginação de memória, a cache de tradução de endereços de memória (Translation Lookaside Buffer, TLB), que é exemplificada na figura junto,

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

A) acelera o acesso à memória porque mantém informação essencial de um conjunto de registos da Tabela de Páginas.

B) acelera o acesso à memória porque substitui integralmente a Tabela de Páginas nos acessos de leitura.

C) é especialmente eficaz na aceleração do acesso à memória quando se dá uma "falha de página" (page fault).

Solução: Opção A ☒

Justificação: TLB (Translation Lookaside Buffer) é um pequeno dispositivo de hardware que se encontra dentro do MMU. Serve para mapear endereços virtuais a endereços físicos sem terem que passar pela tabela de páginas. Ter de passar pela tabela de páginas é um problema que afeta a performance, pois todas as referências à memória têm de passar pela tabela de páginas. Com o TLB é possível acelerar o processo.

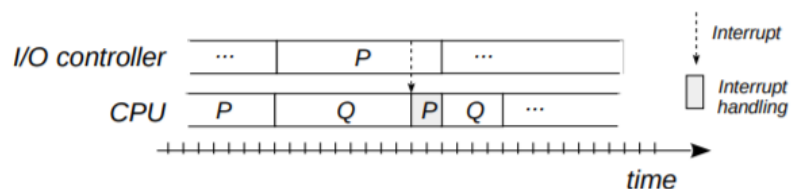
27. Os dispositivos de Entrada/Saída (I/O) dos computadores modernos costumam

- A) ter uma componente de software de controlo.
- B) poder aceder facilmente ao processador principal.
- C) ser do tipo bloco (block devices), de acesso aleatório

Solução: Opção A ☒

Justificação:

28. Na figura junto (tirada das folhas de apoio às aulas teóricas), é patente a vantagem do acesso a dispositivos de Entrada/Saída (I/O) usando interrupções.



- A) Sim, porque é o dispositivo que fica com todo o trabalho de Entrada/Saída.
- B) Sim, porque o processador precisa ser usado por outro processo mais prioritário.
- C) Sim, porque outro processo pode aproveitar o processador, sem prejuízo do acesso de Entrada/Saída.

Solução: Opção C ☒

Justificação:

29. O “tick” do relógio interno de um computador está associado a

- A) um segundo.
- B) um sinal de alarme.
- C) uma interrupção de relógio.

Solução: Opção C ☒

Justificação:

30. Os discos de estado sólido (SSD, Solid State Drive) são muitas vezes preferidos a discos clássicos, magnéticos (HDD, Hard Disk Drive) porque, para igual capacidade de armazenamento,

- A) são mais rápidos.
- B) são mais baratos.
- C) são mais programáveis.

Solução: Opção A ☒

Justificação:

31. Dependendo do contexto, todos os nomes de ficheiro que se apresentam a seguir podem ser utilizados para identificar o mesmo ficheiro. Qual deles é um nome absoluto?

- A) /lib/dict
- B) ./lib/dict
- C) ../lib/dict

Solução: Opção A ☒

Justificação: Em sistemas Unix, um nome absoluto começa com “/”.

- /usr/lib/dict

Um nome relativo não começa com “/”. Ou seja, os seguintes exemplos são nomes relativos:

- `lib/dict` (o ficheiro está no diretório “lib” que se encontra no diretório atual)
- `./dict` (o ficheiro está no diretório atual. É o mesmo que pôr apenas `dict`)
- `../lib/dict` (o ficheiro está no diretório 1 nível acima do diretório atual - o diretório “pai”)
- `../../dict` (o ficheiro está no diretório 2 níveis acima do diretório atual - o diretório “avô”)

32. Os ficheiros costumam ser guardados nos discos em blocos de igual tamanho. O valor do bloco afecta a rapidez de leitura e o aproveitamento do espaço de armazenamento.

A) Sim, pois com grandes blocos o espaço é melhor aproveitado.

B) Sim, pois com grandes blocos as leituras são mais rápidas.

C) Sim, pois com grandes blocos as leituras são mais rápidas e o espaço melhor aproveitado.

Solução: Opção B ☒

Justificação: Observando o seguinte gráfico, é possível ver que com grandes blocos (1MB), a rapidez de leitura (*data rate*) é alta, mas o espaço não é bem aproveitado.

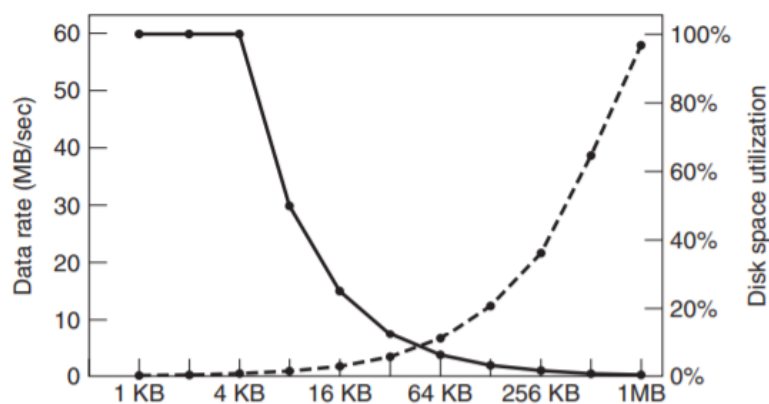


Figure 4-21. The dashed curve (left-hand scale) gives the data rate of a disk. The solid curve (right-hand scale) gives the disk-space efficiency. All files are 4 KB.

33. Os directórios têm, no sistema de ficheiros, uma estrutura interna específica.

A) Sim, e é semelhante à dos ficheiros regulares estruturados.

B) Sim, por isso é que têm chamadas ao sistema específicas, como `opendir()`.

C) Não, porque são apenas sequências de bytes.

Solução: Opção B ☒

Justificação: Os directórios são um tipo de ficheiro e, portanto, têm uma estrutura interna específica que resulta em chamadas ao sistema específicas a esta, como por exemplo `opendir()`.

A opção A está incorreta, pois embora os directórios e os ficheiros regulares sejam ficheiros, eles são diferentes.

A opção C está incorreta. Sequências de bytes é um tipo de estrutura que se aplica aos ficheiros regulares e não aos directórios.

34. Em Unix, um nó de indexação (i-node) contém os atributos de um ficheiro e os seus blocos de dados.

A) Sim, o i-node contém o nome do ficheiro, a data da sua criação, os seus blocos de dados, etc.

B) Só será assim se o ficheiro for pequeno; se for grande, o i-node não tem blocos de dados.

C) Não, o i-node além dos atributos só tem endereços que conduzem aos blocos de dados.

Solução: Opção C ☒

Justificação: Na seguinte figura verifica-se que um i-node contém os atributos do ficheiro e endereços dos blocos do disco. Os blocos de dados encontram-se então em disco e no i-node guardamos apenas os seus endereços relevantes àquele ficheiro.

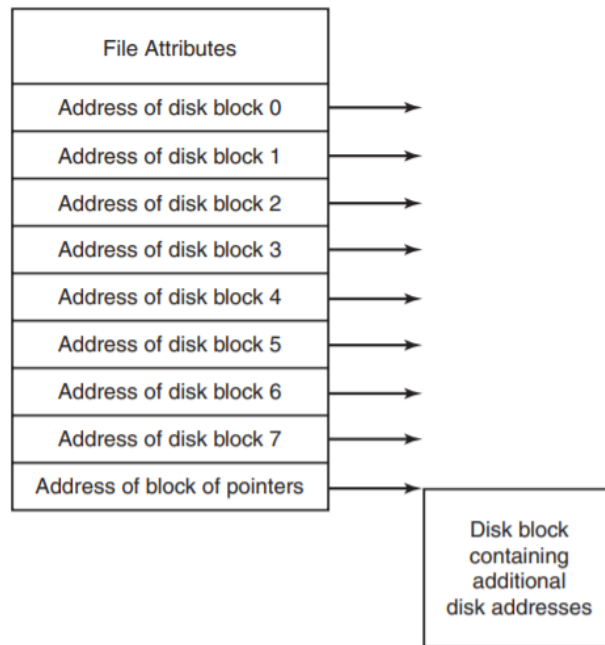


Figure 4-13. An example i-node.

35. A função `search()`, cujo esqueleto é apresentado a seguir, é a parte essencial de um programa que, recursivamente, percorre a directoria corrente (ou de trabalho) e as suas sub-directorias identificando todos os links simbólicos e apresentando-os, pelo nome simples, na saída padrão.

```

void search() {
    DIR *dir = opendir(".");
    struct dirent *f;
    while ((f = ( .1 )) != NULL) {
        if ( .2 )
            printf("%s\n", f->d_name);
        else if (f->d_type == DT_DIR && strcmp(f->d_name, "..") && strcmp(f->d_name, "."))
            { .3 }
    }
} //search()

```

Cada local assinalado por '`.n`' (por exemplo, `.2`) deve ser preenchido com código apropriado, que deverá escolher, de entre as opções a seguir mostradas para cada um.

SUGESTÃO IMPORTANTE: tente completar o código de cada local antes de escolher de entre os extractos mostrados!

`.1` :

- A) `open(dir);`
- B) `readdir(dir);`
- C) `read(dir);`

Solução: Opção B 

Justificação: Dentro do while, o objetivo é ler os conteúdos do directório “dir” para posteriormente verificar que tipo de conteúdo foi lido.

A opção A está incorreta, pois a função open() é usada para abrir ficheiros e não directórios.

A opção C está incorreta pela mesma razão: a função read() é usada para ler os conteúdos de um ficheiro e não de um directório.

A opção B é a correta, pois é a que lê as entradas do directório.

2 :

A) f->d_type == DT_LNK;

B) f->d_type != DT_LNK;

C) f->d_type && DT_LNK;

Solução: Opção A 

Justificação: No formulário fornecido é possível ver que DT_LNK identifica que o directório é um link simbólico. O enunciado pede para identificar “todos os links simbólicos e apresentá-los, pelo nome simples, na saída padrão”. Tendo em conta que dentro da instrução do if temos um printf do nome simples, podemos concluir que a condição do if serve para verificar se o sub-directório é um link simbólico.

.3 :

A) chdir(f->d_name); search(); chdir(dir);

B) chdir(f->d_name); search(); chdir("../");

C) chdir(f->d_name); search(); chdir("./");

Solução: Opção B 

Justificação: Protótipo da função chdir:

```
int chdir(const char *pathname);
```

Muda o directório por defeito corrente (onde se encontra atualmente) para pathname.

Logo, ao fazer `chdir(f->d_name)` estamos a entrar no sub-directório que inspecionamos e recursivamente inspecionamos também os seus conteúdos. Depois, queremos voltar para o directório onde estávamos para continuar de onde partimos: `chdir("..")`. Sendo que `".."` identifica o directório-pai.

A opção C está incorreta, pois apenas permanecemos no mesmo directório. Nada muda com a instrução `"."`.

A opção A está incorreta pois `chdir()` não aceita um directório como parâmetro, mas sim uma *string*.

NOTA: `chdir` = `cd` = change directory

36. Considere os seguintes excertos simplificados de código, que respeitam a uma aplicação cliente-servidor do género da usada no 2º mini-projecto. Suponha também que não se dão situações inesperadas de erro (além do que estiver previsto nas perguntas).

```
// servidor
mkfifo(fifoname, 0666);
fifo = open(fifoname, O_RDONLY);
while (read(fifo, &msg, sizeof(msg)) > 0)
    { ... // processa msg }
close(fifo);
unlink(fifoname);
```

```
// cliente
fifo = open(fifoname, O_WRONLY|O_NONBLOCK
);
    { ... // constrói msg }
write(fifo, &msg, sizeof(msg));
close(fifo);
```

a) Suponha que o servidor arrancava primeiro, uns segundos antes do cliente. Quando o cliente arrancasse,

- A) o servidor estaria bloqueado no `mkfifo()`.
- B) o servidor estaria bloqueado no `open()`.
- C) o servidor estaria bloqueado no `read()`.

Solução: Opção B 

Justificação: A função `open()` no servidor tem as seguintes *flags*:

- `O_RDONLY`: abre o ficheiro apenas para leitura
- `O_NONBLOCK`: Se estiver ativo, retorna sem qualquer atraso. Se não estiver ativo, bloqueia a thread (o servidor) até outra thread abrir o ficheiro para escrita. (Isto é válido quando `O_NONBLOCK` está associado com `O_RDONLY`).

Ora, aqui a flag `O_NONBLOCK` não está ativa e, portanto, bloqueia no `open()` até que o servidor abra o ficheiro para escrita.

b) Se a FIFO não existir quando o `open()` no cliente é executado

- A) o programa dá imediatamente erro no `open()`.
- B) o programa dá imediatamente erro no `write()`.
- C) o programa bloqueia no `open()` até o FIFO ser criado.

Solução: Opção A ☒

Justificação: A função `open()` no cliente tem as seguintes *flags*:

- `O_WRONLY`: abre o ficheiro apenas para escrita
- `O_NONBLOCK`: Se estiver ativo, retorna um erro se nenhum processo tiver o mesmo ficheiro aberto para escrita. Se não estiver ativo, bloqueia a thread (o cliente) até outra thread abrir o ficheiro para leitura. (Isto é válido quando `O_NONBLOCK` está associado com `O_WRONLY`).

Se o FIFO não existe significa que o servidor ainda não foi executado (sendo que é o servidor o responsável por criar esse FIFO). No entanto, isso até é irrelevante, pois o outro thread não abre o FIFO para escrita (apenas para leitura - `O_RDONLY`). Assim, podemos assumir que nenhum outro processo tem o mesmo ficheiro aberto para escrita e, portanto, o `open()` retorna um erro.

c) Suponha que vários clientes (do tipo mostrado) eram lançados concorrentemente. Para garantir que o servidor não recebia mensagens de conteúdo misturado,

- A) bastava que `sizeof(msg)` fosse inferior a `PIPE_BUF`.
- B) não era preciso ter-se um cuidado especial.
- C) era sempre necessário usar-se mutexes no acesso ao FIFO.

Solução: Opção A ☒

Justificação: Sempre que as escritas/leitura de um *file descriptor* forem inferiores a `PIPE_BUF` não é necessário qualquer mecanismo de sincronização para assegurar que as leituras/escritas sejam atômicas.

“says that `write(2)`s of less than `PIPE_BUF` bytes must be atomic: the output data is written to the pipe as a contiguous sequence.”

(<https://man7.org/linux/man-pages/man7/pipe.7.html>)

37. Um sistema operativo de tempo-real é usado especialmente em aplicações que processam dados à medida que lhe são entregues, sem atrasos devidos a armazenamento temporário (e.g. em buffers).

A) Assim é porque o processamento desses dados tem de ser feito num tempo específico.

B) Assim é porque o utilizador de tais aplicações não quer estar à espera: por exemplo, se liga um interruptor, a lâmpada do tipo "smart" tem de acender de imediato.

C) Assim é porque tais aplicações operam com equipamento muito simples sem provisão para memória temporária (buffers).

Solução: Opção A ☒

Justificação: A opção A é, basicamente, a definição de um RTOS (Real-Time Operating System).

Segundo a Wikipedia, "Um RTOS não tem que ter necessariamente um elevado débito nas saídas, ou um elevado número de saídas, no entanto, tem que garantir que certas tarefas sejam executadas em um determinado intervalo de tempo. Um RTOS é mais eficaz e é mais valorizado pela forma previsível e rápida na resposta a um evento, do que pela quantidade de dados que processa."

38. Das muitas ameaças à segurança de utilização de um computador, a do tipo "cavalo de Tróia" pode ser evitada pelo sistema operativo.

A) Sim, é certo, porque o sistema operativo pode detectar a instalação do "cavalo".

B) Talvez possa ser mitigada, caso o administrador faça uma configuração adequada do sistema.

C) Não, este é um tipo de ameaça em que só o utilizador pode evitar o problema.

Solução: Opção B ☒

Justificação: Com "configuração adequada do sistema" significa, por exemplo, a instalação de antivírus.

39. Num terminal de texto em Unix, deu o comando “ls -l” e obteve

```
-rwxr-xr-x 1 user grp 8335 jan 18 2019 test1
```

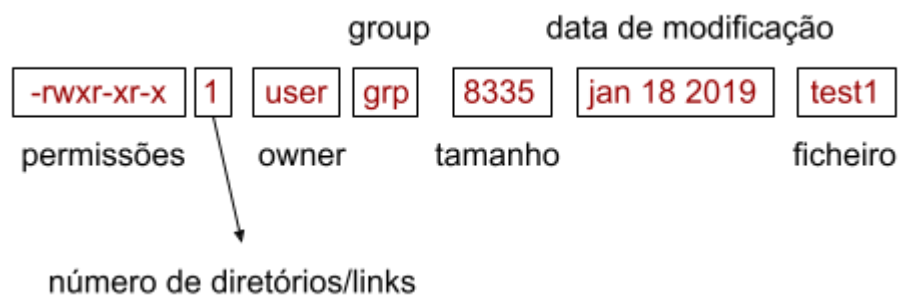
A) Concluiu que o sistema devia usar algo semelhante a listas de controlo de acesso (ACL) na protecção dos ficheiros.

B) Concluiu que o sistema devia usar algo semelhante a credenciais de acesso (capabilities) na protecção dos ficheiros.

C) Nada lhe permitiu concluir sobre a forma como o sistema guardava a Matriz de Protecção.

Solução: Opção A ☒

Justificação: Primeiro, é importante perceber o significado de cada ponto do observado no terminal de texto.



Listas de controlo de acesso (ACL) guardam para cada ficheiro/objeto, uma lista com os domínios (user, group) e as suas respectivas permissões.

Por outro lado, credenciais de acesso (capabilities) guardam para cada domínio (user, group), uma lista com os ficheiros/objetos e respetivas permissões.

Aqui, podemos ver que para o ficheiro “test1”, temos permissões associadas a “user”, a “group” e a “others”, pelo que o sistema do Unix deve usar algo semelhante a ACL.

40. Usamos Unix/Linux na disciplina porque, entre outras coisas:

A) Tem uma interface "seca" para o utilizador, o que é um bom treino para um futuro Engenheiro Informático.

B) Tem uma interface programática excepcionalmente simples e reconfigurável, característica de um sistema monolítico.

C) Pode ser explorado sem restrições de licenciamento e de acesso aos pormenores de implementação.

Solução: Opção C ☒